



Projektplan G2T2 Memori

Softwarearchitektur WS2022/23

CHI Boon-Chung

RUFINATSCHA Daniel

MAIRHOFER Simon

ROED Hannes

HOFER Lukas

Einführung und Ziele	2
Aufgabenstellung	3
Qualitätsziele	3
Stakeholder	3
Randbedingungen	4
Kontextabgrenzung	5
Fachlicher Kontext	5
Technischer Kontext	6
Lösungsstrategie	7
UI-Sketch	9
Verteilungssicht	11
Querschnittliche Konzepte	12
Architekturentscheidungen	13
Glossar	14

Einführung und Ziele

Das Projekt “Memori” ist eine Webplattform und soll das Lernen nach dem Spaced Repetition und Active Recall Prinzip ermöglichen.

Dazu gehören:

- Erstellen von Decks und Lernkarten, auf denen Frage und Lösung in geeigneter Form separat auf Vorder- und Rückseite gespeichert und in einer Lernansicht präsentiert werden.
- Implementierung eines geeigneten Spaced Repetition Algorithmuses, der die Lernkarten in wissenschaftlich erwiesenen idealen Zeitabständen zur Wiederholung präsentiert.
- Die Webapplikation soll es ermöglichen, erstellte Decks mit anderen Studierenden zu teilen.

Aufgabenstellung

Verweis auf: Einführung und Ziele

Verweis auf Dokument: Projektbeschreibung_WS22.pdf

Qualitätsziele

ID	Priorität	Anforderung	Erklärung
Q-1	1	Sicherheit	Mittels JSON Web Token (JWT). Das JWT ermöglicht den Austausch von verifizierbaren Claims. Sämtliche authentifizierungsrelevanten Informationen können im Token übertragen werden und die Sitzung muss nicht zusätzlich auf einem Server gespeichert werden . (Siehe unter Source_Code_Dokumentation/Security)
Q-2	2	Benutzerfreundlichkeit	
Q-3	3	Flexibilität	
Q-4	4	Performanz	
Q-5	5	Korrektheit	

Stakeholder

Rolle	Ziel	Erwartungshaltung
Frontend	Entwicklung des frontends	
Backend	Entwicklung des backends	
Datenbankentwicklung	Entwicklung der Datenbank	
Memori-Nutzer	Möchte die Webplattform als User verwenden.	Nutzer haben an der Architektur des Systems kein Interesse. Sie wollen mithilfe der Webplattform mittels Spaced Repetition und Active Recall ihren Lernerfolg steigern.
Administrator	Möchte die Webplattform als Administrator nutzen.	Administratoren haben kein Interesse an der Architektur des Systems. Sie wollen lediglich die Webplattform leiten. User und Decks, die gegen Richtlinien verstoßen, können durch den Administrator blockiert bzw. entfernt werden.
Auftraggeber	Möchte eine Dokumentation des Projekts	Auftraggeber haben Interesse an der gesamten Architektur des Webapplikation.

Randbedingungen

RB	Ziel
RB-1	Die Webapplikation soll bis spätestens den 30. Jänner 2023 fertig entwickelt sein.
RB-2	Die Plattform soll zwei Benutzerrollen unterstützen: Lernende und Administratoren.

RB-3	Die Plattform soll in aktuellen Browsern lauffähig sein.
RB-4	Alle Listenansichten sollen filter- und sortierbar sein.
RB-5	Auf unerwartete Systemzustände (Exceptions) soll das System angemessen reagieren und dem Nutzer entsprechende Fehlermeldungen zurückliefern.
RB-6	Das System soll benutzerfreundlich (Usability) gestaltet sein.
RB-7	Das System soll multiuserfähig sein: Es gibt Lernende und Administratoren. Jeder User des Systems soll mit Username, Passwort (verschlüsselt), Vorname, Nachname, Rolle und Emailadresse gespeichert werden. User müssen sich immer erfolgreich anmelden, um Zugriff auf das System zu erhalten.
RB-8	Das Löschen von Entitäten (Nutzer, Decks, ...) soll unterstützt werden. Dabei ist zu beachten, dass das Löschen von bestimmten Entitäten weitere Auswirkungen auf das System haben kann (z.B.: wenn ein veröffentlichtes Deck gelöscht werden soll). Hierfür sollen geeignete "Lösch-Policies" definiert und umgesetzt werden.

Kontextabgrenzung

- Um E-Mails mit der Webapplikation versenden zu können, wird EmailJS verwendet. Dadurch kann eine E-Mail direkt vom Code aus verschickt werden.

Fachlicher Kontext

Nachbar	Beschreibung
Benutzer	Jeder User muss sich zuerst bei der Login Page mit einem Passwort und einem Benutzernamen anmelden. Ein Benutzer kann Decks erstellen und diese auch mit Lernkarten füllen. Es besitzt auch die Möglichkeit die Decks zu lernen und Decks von anderen Autoren zu implementieren.
Administrator	Ein Administrator kann über sein Interface eine Liste aller Benutzer sehen und diese jederzeit löschen, neue Benutzer hinzufügen oder Benutzer bearbeiten. Der Admin kann dann auch entscheiden ob der neue Benutzer auch ein Administrator sein soll oder ob er nur ein normaler Benutzer sein soll. Die Administratoransicht zeigt auch alle erstellten Decks in einer Liste an, die der Administrator beliebig 'Sperren' kann. Dies hat zur Folge dass alle Nutzer dieses Decks,

	einschließlich der Erstellers, keinen Zugriff auf die Lernansicht und Detailansicht dieses Decks haben.
--	---

Technischer Kontext

	Beschreibung
importierte Decks	Die Importierten Decks haben die Funktion eine Detail-Ansicht aller Karten zu öffnen und das Deck vom eigenen Profil zu löschen. Um Decks zu importieren, wird eine Suchfeld aufgerufen, wo man nach den Namen eines Decks suchen kann. Neben dem Namen wird noch eine kurze Beschreibung des Decks angezeigt.
selbst erstellte Decks	Auch bei den selbst erstellten Decks, kann eine Detailansicht geöffnet werden. Das Deck kann auch veröffentlicht werden, somit ist es auch für andere Benutzer sichtbar. Beim Löschen der selbst erstellten Decks wird das Deck im eigenen Profil und im Profil der User, die dieses Deck importiert haben gelöscht. Auf allen Decks wird zusätzlich noch angezeigt, wie viele der Karten noch nie gelernt/angezeigt wurden und die Anzahl der Karten die zu wiederholen sind.
Detailansicht	Die Detailansicht zeigt die Gesamten Karteikarten eines Decks in Tabellarischer Form an. In der Detailansicht ist es möglich eine Karteikarte zu bearbeiten/umschreiben, einzelne Karteikarten zu löschen, eine neue Karte hinzuzufügen und die Karte verkehrt herum anzeigen zu lassen (Vorderseite und Rückseite werden getauscht). Eine neue Karte hinzufügen wird mithilfe eines pop-up-fensters realisiert.
Karteikarte	Eine Karteikarte/Karte besteht aus einer Vorderseite und einer Rückseite. Auf der Vorderseite soll das zu Abzufragende geschrieben sein. Auf der Rückseite wird die Lösung dargestellt. Zusätzlich gibt es für jede Karte die Möglichkeit sie auch umgekehrt in der Lernansicht anzeigen zu lassen (Die Antwort zuerst).
HOME-Seite	Die Home - Seite wird direkt nach dem Login aufgerufen. Sie wird in 2 Teile gegliedert: In der oberen Hälfte sind alle eigenen Decks gemeinsam mit einem Button zum kreieren neuer Decks gelistet. In der zweiten unteren Hälfte sind alle importierten Decks gemeinsam mit einem Button zum importieren neuer Decks gelistet.

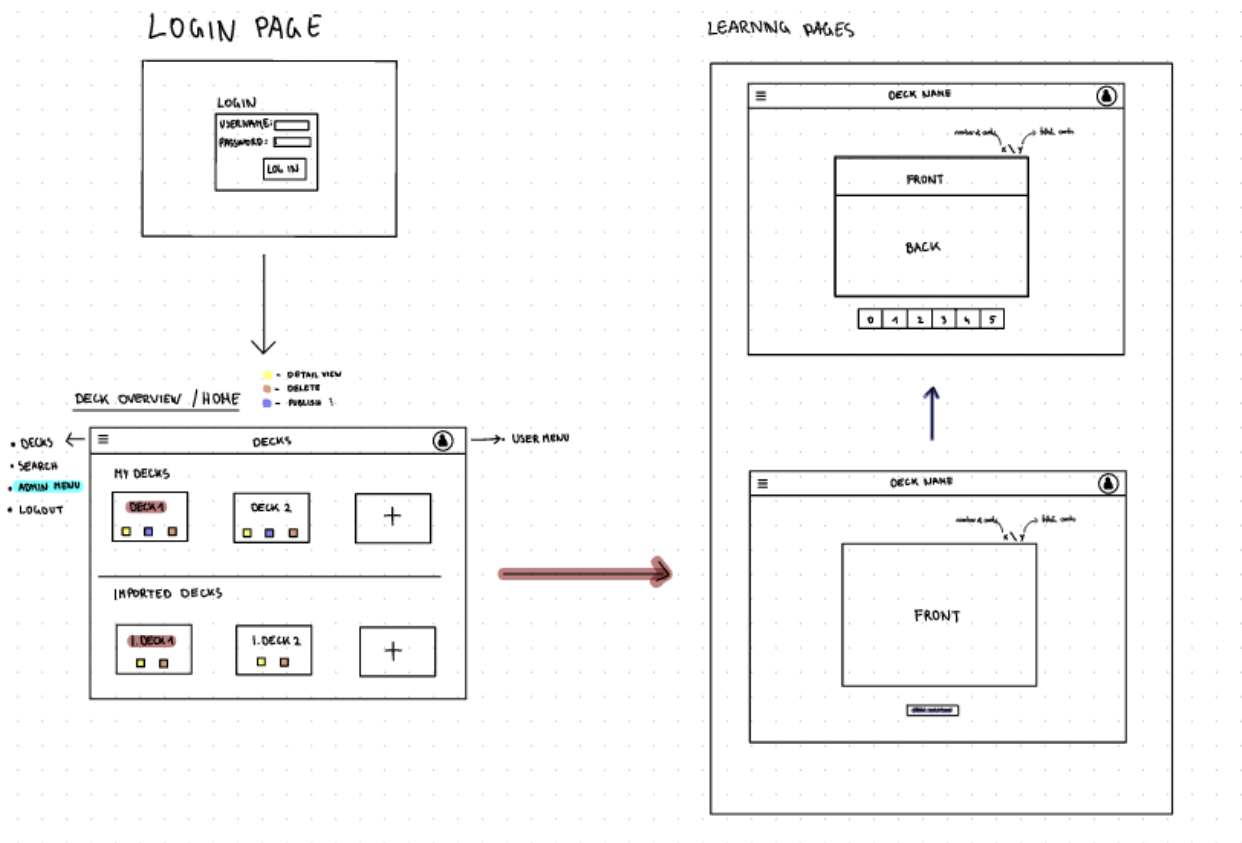
Lernansicht	Hier werden zuerst die zu wiederholenden Karten und dann die noch nie angezeigten Karten nacheinander angezeigt. An der oberen rechten Ecke werden die Anzahl der Karten, die nochmals wiederholt werden müssen und die Anzahl der neuen Karten angezeigt (z.B. 5/30). Für die Lernansicht wird zu Beginn nur die Vorderseite der ersten Karte angezeigt. Durch klicken eines Buttons wird dann die Rückseite, die die Antwort enthält angezeigt. Die Vorderseite wird oberhalb der Antwort immer noch in Kleinformat angezeigt. Nach dem klicken des Buttons für die Lösung ist es möglich eine Bewertung von 0-5 abzugeben, die bedeuten ob man richtig beantwortet hatte und wie schnell man dies gemacht hat. Die Exakte Bedeutung der Skala kann mit einem zusätzlich Button über ein pop-up Fenster aufgerufen werden. Nach dem bewerten wird mit der nächsten Karte fortgefahren. Zusätzlich kann man jederzeit von der Lernansicht eines Decks zurück zur Deck-Übersicht gelangen. Sollte man das komplette Deck fertig gelernt haben, wird man über den Erfolg informiert und man gelangt über einen Button zurück zur Deckübersicht.
-------------	---

Lösungsstrategie

- **Technologieentscheidung:** React als frontend (Siehe Architekturentscheidungen)
- **Spaced Repetition und Active Recall Algorithmus:** (Siehe Projektbeschreibung_WS22.pdf Seite 3 und 4)
- Verwendete Datenbank: H2-Datenbank In-Memory

- **Speicherung der Decks:** Zur Speicherung der Decks wird die bereits vorkonfigurierte H2 in-memory DB verwendet. Alle Decks werden nur einmal in der Datenbank abgespeichert. Die veröffentlichten Decks werden global gültig und sind somit für alle User sichtbar. Die Decks und der Lernfortschritt werden separat abgespeichert. Wird ein Deck von einem Benutzer importiert, wird ein neuer Fortschritt für das Deck angelegt. Der neue Fortschritt besitzt eine Referenz auf das importierte Deck. Sollte der Administrator ein Deck sperren, ist es für niemanden mehr einsehbar, bis das Deck wieder freigegeben wird.

UI-Sketch



DECK NAME

FRONT

BACK

0 1 2 3 4 5

number of cards
add / add

FINISH PAGE

DECK NAME

CONGRATS "USERNAME"

You completed the deck

RETURN HOME

DECK OVERVIEW / HOME

DECKS

MY DECKS

DECK 1

DECK 2

+

IMPORTED DECKS

1. DECK 1

1. DECK 2

+

DETAILVIEW

DECK NAME

DESCRIPTION

VORDERSEITE	RÜCKSEITE			
QUESTION	ANSWER	☒	☐	☐
QUESTION	ANSWER	☒	☐	☐
QUESTION	ANSWER	☒	☐	☐

ADD NEW CARD

RETURN BACK

SEARCH

SEARCH DECKS

Q

TITLE	DESCRIPTION	IMPORT
...	...	button

ONLY IF YOU ARE ADMIN

ADMIN MENU

USERNAME	EMAIL		

+ NEW USER

DECK ID	TITLE	CREATED BY	EMAIL	

POP UP WINDOWS:

ADD NEW DECK:

TITLE:

TITLE

DESCRIPTION:

DESCRIPTION

CANCEL

CREATE

ADD NEW CARD:

DECK NAME

FRONT:

QUESTION

BACK:

ANSWER

ALLOW REVERSE: ☒

CANCEL

CREATE

ADD NEW USER:

CREATE NEW USER

USERNAME

PASSWORD

FIRST NAME

LAST NAME

ROLE

☐ ADMIN ☐ USER

EMAIL

CANCEL

CREATE

EDIT CARD IN DETAILVIEW:

DECK NAME

FRONT:

QUESTION

BACK:

ANSWER

ALLOW REVERSE: ☒

CANCEL

UPDATE

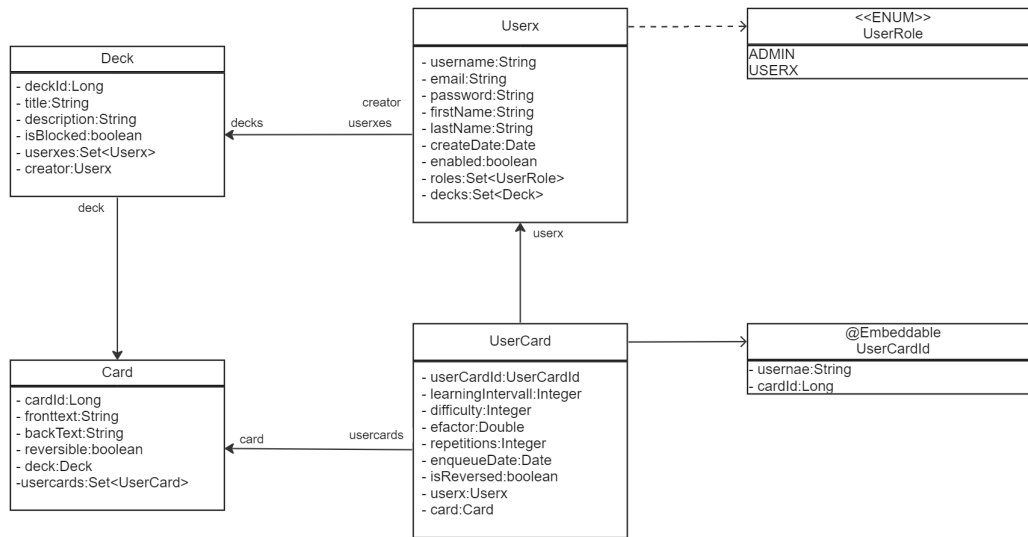
DELETE CARD:

DO YOU WANT TO DELETE THIS CARD PERMANENTLY?

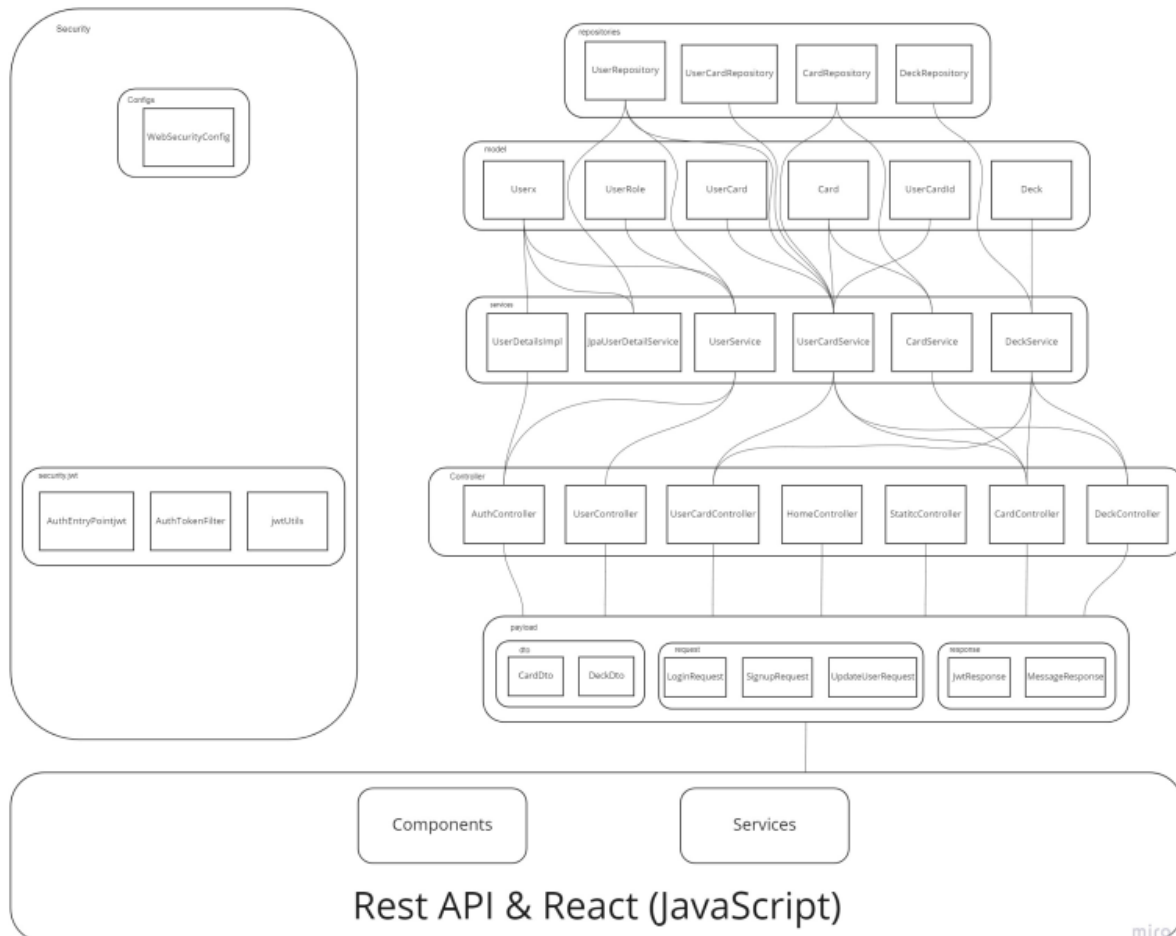
NO

YES

Verteilungssicht



Verteilungsdiagramm



Querschnittliche Konzepte

- Eingesetzte Pattern:
 - Data Transfer Object (Entwurfsmuster)
- Eingesetzte Architekturmuster:
 - Model-View-Controller
 - 3 Schichten Architektur

Architekturentscheidungen

Entscheidung - React als UI Framework anstelle von Java ServerFaces (JSF)

Als Grundstruktur für die Frontend-Webentwicklung verwenden wir als UI Framework React anstelle von Java ServerFaces.

Entscheidungskriterien:

- Für eine eventuelle Weiterentwicklung zu einer mobile app aufgrund von React Native.
- Einfache Erstellung dynamischer Anwendungen: React benötigt weniger Code und bietet mehr Funktionalitäten.

Entscheidung - Speicherung der Decks und des Lernfortschrittes

Die Decks und der Lernfortschritt werden separat abgespeichert. Wird ein Deck von einem Benutzer importiert, wird ein neuer Fortschritt für das Deck angelegt. Der neue Fortschritt besitzt eine Referenz auf das importierte Deck. (Siehe auch Lösungsstrategien, Speicherung der Decks.)

Entscheidungskriterien:

- Da Decks samt Lernkarten auch veröffentlicht werden können, besitzt ein Deck auch verschiedene Lernfortschritte der verschiedenen Benutzer. Aus diesem Grund wird der Lernfortschritt separat vom Deck abgespeichert.

Source Code Dokumentation

Entitäten

Die Entitäten befinden sich im Package model.

Es existieren 6 Entitäten:

- User

- Diese Entität beschreibt den Benutzer.

```
@{...}
private Set<UserRole> roles;
@{...}
private Set<Deck> decks = new LinkedHashSet<>();
@{...}
private Set<UserCard> userCards = new LinkedHashSet<>();
```

-
- Ein Benutzer enthält eine Menge von Decks und Lernkarten und besitzt eine UserRole. Diese beschreibt, ob der Benutzer ein Administrator oder ein normaler Benutzer der Webanwendung ist.

- Deck

- Diese Entität beschreibt das Deck.

```
@{...}
private Set<Card> cards = new LinkedHashSet<>();
@{...}
private Set<Userx> userxes = new LinkedHashSet<>();
```

-
- Ein Deck enthält eine Menge an Lernkarten und eine Menge an Benutzern, die dieses Deck importiert haben.

- Card

- Diese Entität beschreibt eine Lernkarte.

```
@{...}
private Set<UserCard> userCards = new LinkedHashSet<>();
```

-
- Eine Lernkarte enthält eine Menge aller Beziehung zwischen User und Card.

- UserCard

- UserCard ist eine m:n Relation zwischen User und Card. Die Relation besitzt eigene Attribute. UserCard wird verwendet, um verschiedene Lernfortschritte von verschiedenen Benutzern (aufgrund der Möglichkeit von importierten Decks) zu speichern.

- ```

@Column(name = "learning_intervall")
private Integer learningIntervall = 0;
2 usages
@Column(name = "efactor")
private Double efactor = 2.5;
2 usages
@Column(name = "repetitions")
private Integer repetitions = 0;
2 usages
@Column(name = "enqueue_date")
@JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM-dd'T'HH:mm:ss.SSSXXX")
private Date enqueueDate = Date.valueOf(LocalDate.now());

```
- ```

4 usages
@ManyToOne
@MapsId("username")
@JoinColumn(name = "userx_username")
@JsonIgnore
private Userx userx;

4 usages
@ManyToOne
@MapsId("cardId")
@JoinColumn(name = "card_card_id")
@JsonIgnore
private Card card;
      
```

- **UserCardId**

- UserCardId oder auch UserDeck ist eine m:n Relation zwischen Userx und Deck. Die Relation besitzt keine eigenen Attribute und wird verwendet, um Benutzern mehrere Decks zur Verfügung zu stellen. Das gleiche Deck kann auch von mehreren Benutzern implementiert werden, darum m:n.

- ```

6 usages
@Column(name = "username")
private String username;

6 usages
@Column(name = "card_id")
private Long cardId;

```

## Logik

Die Logik befindet sich im Package services.

Services trennen die Geschäftslogik von der Präsentationsschicht.

- **CardService**
  - Mithilfe des CardServices können Karten geladen, gespeichert, gelöscht oder auch umgedreht werden (Vorderseite ist die neue Rückseite und umgekehrt).
- **DeckService**
  - Mithilfe des DeckServices können Decks geladen, gespeichert, gelöscht und veröffentlicht werden. Zudem existieren noch weitere Methoden, um eine

Liste an Decks anzufragen, die bestimmte Kriterien erfüllen. (z.B. `getDecksByCreator()` liefert eine Liste aller Decks die User xy erstellt hat.)

```
public List<Deck> getAllDecks() { return deckRepository.findAll(); }
public Deck loadDeck(Long deckId) { return deckRepository.findById(deckId); }
public Deck saveDeck(Deck deck) { return deckRepository.save(deck); }
public void deleteDeck(Long deckId) {...}
public void publishDeck(Long deckId) {...}
public void togglePublishDeck(Long deckId) {...}
public List<Deck> getDecksByTitle(String title) { return deckRepository.findByTitleContaining(title); }
public List<Deck> getDecksByCreator(String creator) { return deckRepository.findByCreator_username(creator); }
public List<Deck> getPublishedDecks() { return deckRepository.findByPublishedIsTrue(); }
public Optional<Deck> getDeckById(Long deckId) { return deckRepository.findById(deckId); }
public Boolean userIsCreator(Long deckId, String username) {...}
public void toggleIsBlocked(Long deckId) {...}
public List<Deck> getImportedDecks(String username) {...}
public List<Deck> getPublishedDecksNotImported(String username) {...}
public Boolean isImported(String username, Long deckId) {...}
```

- 
- JpaUserService
  - Java Persistence API ermöglicht es Java-Objekte zu mappen, zu speichern, zu suchen und zu löschen, indem es Methoden bereitstellt, die auf Java-Objekte anstatt auf relationalen Tabellen angewendet werden.
  - UserService stellt Methoden bereit, um Detaillierte Informationen über einen Benutzer nur mithilfe des Benutzernamens zu erhalten.
- UserCardService
  - Der UserCardService stellt verschiedene Methoden bereit: `getAllUserCardsByUser()`, `getNextCard()`, `setEnqueueDate()`, `initQueue()`, `addCardsToUser()`, `removeCardFromUser()`. Die Methoden `initQueue` und `setEnqueueDate` implementieren den Lernalgorithmus

```
public int setEnqueueDate(Integer difficulty) {
 UserCard userCard = queue.poll();
 userCard.setRepetitions(userCard.getRepetitions() + 1);
 if (difficulty < 3) {
 userCard.setLearningInterval(1);
 }
 if (difficulty < 4) {...}
 if (difficulty > 2) {...}
 userCardRepository.save(userCard);
 return queue.size();
}
```

```
public int initQueue(String username, Long deckId) {
 queue.clear();
 List<UserCard> userCardList = this.getAllUserCardsByUser(username);
 List<Card> cardListByDeck = cardService.getCardsByDeck(deckId);

 for (Card c : cardListByDeck) {
 for (UserCard uc : userCardList) {
 if (uc.getCard().equals(c) && uc.getEnqueueDate().before(Date.valueOf(LocalDate.now()))
 || uc.getCard().equals(c) && uc.getEnqueueDate().equals(Date.valueOf(LocalDate.now()))) {
 queue.add(uc);
 }
 }
 }

 return queue.size();
}
```

- UserService
  - UserService stellt Methoden für das Manipulieren und Anfragen von UserAttributen.

## REST API

Rest API befindet sich im Package controller.

REST (Representational State Transfer) API (Application Programming Interface)

REST API benutzt HTTP-Methoden wie GET, POST, PUT und DELETE um Daten zu empfangen oder zu manipulieren.

- AuthController
  - Authentifizierung des Benutzers: Benutzer / Administrator

```
@PostMapping("/signin")
public ResponseEntity<JwtResponse> authenticateUser(@RequestBody LoginRequest loginRequest) {...}

@PostMapping("/signup")
@PreAuthorize("hasAuthority('ADMIN')")
public ResponseEntity<MessageResponse> registerUser(@RequestBody SignupRequest signUpRequest) {...}
```

- CardController
  - Controller für CardService
- DeckController
  - Controller für DeckService
- HomeController
  - Der Controller verweist auf die index.html Seite im templates Ordner.

```
@GetMapping("/")
public String index() { return "index"; }
```

- StaticController
  - Der Controller verweist auf die index.html Seite im templates Ordner.

```
@GetMapping(value = {"/admins/*", "/users/*"})
public String indexRedirect() { return "index"; }
```

- UserCardController
  - Controller für UserCardService
- UserController
  - Controller für UserService

## REST API im frontend

REST API im frontend befindet sich unter dem Package frontend.src.services.

Nötige HTTP requests an die Rest-API.

- AuthService

```

○ const API_URL = "http://localhost:8080/api/auth/";

logout() {
 localStorage.removeItem("user");
}

register(username, email, password, firstName, lastName, roles, enabled) {...}

getCurrentUser() {
 return JSON.parse(localStorage.getItem('user'));
}

```

- CardService

```

○ const API_URL = 'http://localhost:8080/api/cards';

getCardsByDeckId(deckId) {
 return axios.get(API_URL + "/allCards/" + deckId, {headers: authHeader()});
}

saveCard(frontText, backText, deckId, reversed) {...}

deleteCard(cardId) {
 return axios.delete(API_URL + "/deleteCard/" + cardId, { headers: authHeader() });
}

loadCard(cardId) {
 return axios.get(API_URL + "/" + cardId, {headers: authHeader()});
}

updateCard(cardId, frontText, backText, reversed, deckId) {...}

```

- DeckService

```

○ const API_URL = 'http://localhost:8080/api/deck/';
○ getDeckList(), loadDeck(), addDeck(), publishDeck(), togglePublishDeck(),
 deleteDeck(), updateTitle(), updateDescription(), getAllDecks(),
 toggleBlockedDeck(), getPublicDecks(), getImportedDecks(),
 getPublishedDecksNotImported(), importDeck(), removeImportedDeck(),
 canEditDeck()

```

- UserCardService

```

○ const API_URL =
 'http://localhost:8080/api/userCards/';
○ initDeckQueue(), getNextCard(), valuateCard()

```

- UserService

```

○ const API_URL = 'http://localhost:8080/api/users/';
○ getUserList(), deleteUser(), updateUser(), getAllEmailByDeckId()

```

## Security

Zur sicheren Übertragung der Daten wird der JSON Web Token (JWT) verwendet. Es ermöglicht die Authentifizierung von Benutzern durch den Austausch von verschlüsselten Token. Der Token enthält die Identität des Benutzers.

- AuthEntryPointJwt



- Die Klasse besitzt ein statisches Logger-Object, das mithilfe der LoggerFactory erstellt wurde und dazu dient, Log-Einträge zu generieren.

```
private static final Logger logger = LoggerFactory.getLogger(AuthEntryPointJwt.class);
```

- Die Methode commence() wird aufgerufen, wenn ein unauthorisierter Zugriffsversuch erfolgt. In diesem Fall kommt es zu einem "Unauthorized error" und es wird ein HTTP-Fehler 401 (Unauthorized) zurück an den Client geschickt.

```
@Override
public void commence(HttpServletRequest request, HttpServletResponse response, AuthenticationException authException)
 throws IOException, ServletException {
 logger.error("Unauthorized error: {}", authException.getMessage());
 response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Error: Unauthorized");
}
```

- **AuthTokenFilter**

- Die doFilterInternal() Methode extrahiert aus dem Token den Benutzer und informiert Spring, dass der Benutzer authentifiziert ist.

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
 throws ServletException, IOException {
 try {
 String jwt = parseJwt(request);
 if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
 String username = jwtUtils.getUserNameFromJwtToken(jwt);

 UserDetails userDetails = userDetailsService.loadUserByUsername(username);
 UsernamePasswordAuthenticationToken authentication = new UsernamePasswordAuthenticationToken(userDetails, credentials: null,
 userDetails.getAuthorities());
 authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

 SecurityContextHolder.getContext().setAuthentication(authentication);
 }
 } catch (Exception e) {
 loggerAuthToken.error("Cannot set user authentication: {}", e.getMessage());
 }

 filterChain.doFilter(request, response);
}
```

- Die Methode parseJwt(HttpServletRequest request) filtert das Keyword "Bearer " heraus da dies für die Übertragung mit HTTP benötigt wird.

```
private String parseJwt(HttpServletRequest request) {
 String headerAuth = request.getHeader("Authorization");

 if (StringUtils.hasText(headerAuth) && headerAuth.startsWith("Bearer ")) {
 return headerAuth.substring(beginIndex: 7);
 }

 return null;
}
```

- **JwtUtils**

- Die Methode generateJwtToken() erstellt ein neues JWT, indem sie Informationen über den aktuellen Benutzer als Claim hinzufügt und es dann signiert.

```

public String generateJwtToken(Authentication authentication) {

 UserDetailsImpl userPrincipal = (UserDetailsImpl) authentication.getPrincipal();

 return Jwts.builder().setSubject((userPrincipal.getUsername())).setIssuedAt(new Date())
 .setExpiration(new Date((new Date()).getTime() + jwtExpirationMs)).signWith(SignatureAlgorithm.HS512, jwtSecret)
 .compact();
}

```

- 
- Die Methode `getUserNameFromJwtToken()` extrahiert den Benutzer aus einem gegebenen JWT.

```

public String getUserNameFromJwtToken(String token) {
 return Jwts.parser().setSigningKey(jwtSecret).parseClaimsJws(token).getBody().getSubject();
}

```

- 
- Die Methode `validateJwtToken()` überprüft die Gültigkeit eines JWT.

```

public boolean validateJwtToken(String authToken) {
 try {
 Jwts.parser().setSigningKey(jwtSecret).parseClaimsJws(authToken);
 return true;
 } catch (SignatureException e) {
 logger.error("Invalid JWT signature: {}", e.getMessage());
 } catch (MalformedJwtException e) {
 logger.error("Invalid JWT token: {}", e.getMessage());
 } catch (ExpiredJwtException e) {
 logger.error("JWT token is expired: {}", e.getMessage());
 } catch (UnsupportedJwtException e) {
 logger.error("JWT token is unsupported: {}", e.getMessage());
 } catch (IllegalArgumentException e) {
 logger.error("JWT claims string is empty: {}", e.getMessage());
 }

 return false;
}

```

-